

DEEP REINFORCEMENT LEARNING

HOMEWORK 1 | VALUE-BASED METHODS

Prof. Mohammad Hossein Rohban
Sharif University of Technology
Spring 2026

Yashar Zafari 404210253

 yaswhar

Due: June 1, 2021



1 – TD Learning with Linear Function Approximation

We consider a linear value function approximator of the form

$$V(s; w) = \phi(s)^\top w.$$

After observing a transition (S_t, R_t, S_{t+1}) , we define the TD error

$$\delta_t = R_t + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t.$$

1.1 (2 Points) TD-SGD Derivation

Starting from the squared TD error loss

$$L_t(w) = \frac{1}{2} (R_t + \gamma \phi(S_{t+1})^\top w - \phi(S_t)^\top w)^2,$$

derive the stochastic gradient descent update for w and show that it has the form

$$w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t).$$

Answer

We are given the squared TD error loss for a single transition (S_t, R_t, S_{t+1}) :

$$L_t(w) = \frac{1}{2} (R_t + \gamma \phi(S_{t+1})^\top w - \phi(S_t)^\top w)^2.$$

Define the TD error as

$$\delta_t(w) = R_t + \gamma \phi(S_{t+1})^\top w - \phi(S_t)^\top w.$$

Then $L_t(w) = \frac{1}{2} \delta_t(w)^2$.

In the **semi-gradient** approach of TD learning, the bootstrap target $R_t + \gamma \phi(S_{t+1})^\top w$ is treated as a constant when differentiating. Hence we differentiate only the part where w appears inside the current prediction $\phi(S_t)^\top w$:

$$\nabla_w \delta_t(w) = -\phi(S_t).$$

The gradient of the loss is therefore

$$\nabla_w L_t(w) = \delta_t(w) \cdot \nabla_w \delta_t(w) = \delta_t(w) \cdot (-\phi(S_t)) = -\delta_t(w) \phi(S_t).$$

Applying stochastic gradient descent with learning rate α :

$$\begin{aligned} w_{t+1} &= w_t - \alpha \nabla_w L_t(w) \big|_{w=w_t} \\ &= w_t - \alpha (-\delta_t(w_t) \phi(S_t)) \\ &= w_t + \alpha \delta_t(w_t) \phi(S_t). \end{aligned}$$

Because $\delta_t(w_t)$ is exactly the TD error δ_t defined in the problem, we obtain the update rule:

$$\boxed{w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t)}.$$

1.2 (1 Points) Update-Rule Comparison

Let us compare this with the TD learning update for finite MDPs, which is

$$V(S_t) \leftarrow V(S_t) + \alpha [R_t + \gamma V(S_{t+1}) - V(S_t)].$$

What are the similarities and differences between these two update rules?

Answer

Similarities:

- Both updates use the same definition of the TD error: $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$, where in the linear case $V(s) = \phi(s)^\top w$.
- Both treat the bootstrap target $R_t + \gamma V(S_{t+1})$ as fixed during the update (semi-gradient approach).
- Both take the form of a stochastic gradient step: new estimate = old estimate $+\alpha \times \delta_t \times$ (a direction vector).

Differences:

- **Tabular TD** maintains a separate scalar parameter for each state (the value $V(s)$). The update modifies **only** the parameter corresponding to the visited state S_t ; values of all other states remain unchanged.
- **Linear TD** maintains a weight vector w of dimension equal to the feature space. The update modifies the components of w corresponding to the non-zero entries of $\phi(S_t)$; for dense or shared feature representations this can change the predicted value of **many** states (since $V(s) = \phi(s)^\top w$ couples states through shared weights).
- Consequently, tabular TD has no interference or generalization between states, whereas linear TD exhibits generalization (and possible interference) across states with similar feature vectors.

1.3 (1 Points) Feature/Weight Dimensions

Consider a finite MDP with $N = |S|$. Choose a feature map ϕ such that any value function of the finite MDP can be represented in the form

$$V(s; w) = \phi(s)^\top w.$$

Specify the dimensions of ϕ and w .

Answer

To represent **any** value function of a finite MDP with $N = |S|$ states using the linear form $V(s; w) = \phi(s)^\top w$, the feature map ϕ and weight vector w must each have dimension at least N . This is because the space of all possible value functions over N states is N -dimensional (one independent value per state). With fewer than N parameters, the linear approximator would be restricted to a lower-dimensional subspace and could not represent arbitrary value functions.

The minimal and natural choice is:

- $\phi(s) \in \mathbb{R}^N$ (an N -dimensional column vector)
- $w \in \mathbb{R}^N$ (an N -dimensional column vector)

A concrete example that achieves full representational power is to set $\phi(s)$ to the one-hot encoding of state s : $\phi(s) = \mathbf{e}_s$, where $\mathbf{e}_s$ has a 1 in the position corresponding to s and 0 elsewhere. Then $V(s; w) = w_s$, and any value function is realized by choosing $w_s = V(s)$. Thus the dimensions are:

$$\phi(s) \in \mathbb{R}^N, \quad w \in \mathbb{R}^N$$

1.4 (2 Points) Tabular-Case Update

What is the update rule for TD with linear value function approximation under this particular choice of ϕ ? Is it the same as or different from the TD learning rule for finite MDPs?

Answer

Choose the feature map $\phi(s) = \mathbf{e}_s$, the one-hot vector of length $N = |S|$, where $\mathbf{e}_s$ has a 1 in the position corresponding to state s and 0 elsewhere. Then:

- $\phi(s) \in \mathbb{R}^N, w \in \mathbb{R}^N$.
- The linear approximation becomes $V(s; w) = \phi(s)^\top w = w_s$, i.e., the s -th component of w directly stores the value estimate for state s .

The TD update for linear function approximation is:

$$w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t),$$

with

$$\delta_t = R_t + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t.$$

Substituting $\phi(S_t) = \mathbf{e}_{S_t}$ and $\phi(S_{t+1}) = \mathbf{e}_{S_{t+1}}$:

- $\phi(S_t)^\top w_t = w_{S_t}$ (the current estimate of $V(S_t)$),
- $\phi(S_{t+1})^\top w_t = w_{S_{t+1}}$ (the current estimate of $V(S_{t+1})$),
- $\phi(S_t)$ is nonzero only in the component corresponding to S_t , so the update modifies only w_{S_t} :

$$w_{S_t} \leftarrow w_{S_t} + \alpha \delta_t,$$

where $\delta_t = R_t + \gamma w_{S_{t+1}} - w_{S_t}$.

Thus the update for the value of state S_t is:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_t + \gamma V(S_{t+1}) - V(S_t)],$$

which is exactly the standard TD learning update rule for finite MDPs.

Conclusion: With the one-hot feature map, the linear TD update is **identical** to the tabular TD update.

2 – Bellman or Bellwoman

Recall that a value function is a $|S|$ -dimensional vector where $|S|$ is the number of states of the MDP. When we use the term V in these expressions as an “arbitrary value function”, we mean that V is an arbitrary $|S|$ -dimensional vector which need not be aligned with the definition of the MDP at all. On the other hand, V^π is a value function that is achieved by some policy π in the MDP.

In the first part of this problem, we will explore some general and useful properties of the Bellman backup operator.

$$(BV)(s) = \max_a \left[r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V(s') \right]$$

$$(B^\pi V)(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s)) V(s')$$

2.1 (2 Points) π -Bellman Contraction

Show that the analogous inequality, $\|B^\pi V - B^\pi V'\| \leq \gamma \|V - V'\|$, holds.

Answer

Step 1: Write the per-state difference. For any state s ,

$$\begin{aligned} (B^\pi V)(s) - (B^\pi V')(s) &= \left[r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V(s') \right] \\ &\quad - \left[r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V'(s') \right]. \end{aligned}$$

The reward term cancels, leaving:

$$(B^\pi V)(s) - (B^\pi V')(s) = \gamma \sum_{s'} p(s'|s, \pi(s)) (V(s') - V'(s')).$$

Step 2: Apply the triangle inequality. Take absolute values and use $|\gamma \sum (\cdot)| \leq \gamma \sum |\cdot|$:

$$|(B^\pi V)(s) - (B^\pi V')(s)| \leq \gamma \sum_{s'} p(s'|s, \pi(s)) |V(s') - V'(s')|.$$

Step 3: Bound the weighted sum by the maximum. For each s' , $|V(s') - V'(s')| \leq \max_{s''} |V(s'') - V'(s'')| = \|V - V'\|_\infty$. Hence,

$$\gamma \sum_{s'} p(s'|s, \pi(s)) |V(s') - V'(s')| \leq \gamma \|V - V'\|_\infty \sum_{s'} p(s'|s, \pi(s)).$$

The transition probabilities sum to 1 for a given s and $a = \pi(s)$:

$$\sum_{s'} p(s'|s, \pi(s)) = 1.$$

Thus,

$$|(B^\pi V)(s) - (B^\pi V')(s)| \leq \gamma \|V - V'\|_\infty.$$

Step 4: Take the maximum over states. The inequality holds for every state s , so the supremum (maximum) over s also satisfies:

$$\max_s |(B^\pi V)(s) - (B^\pi V')(s)| \leq \gamma \|V - V'\|_\infty.$$

By definition of the infinity norm,

$$\|B^\pi V - B^\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty.$$

This is exactly the contraction property required. The factor $\gamma < 1$ guarantees that B^π is a contraction mapping in the supremum norm.

2.2 (2 Points) π -Fixed Point Uniqueness

Prove that the fixed point for B^π is unique. Recall that the fixed point is defined as V satisfying $V = B^\pi V$. You may assume that a fixed point exists.

Answer

Assume there exist two fixed points V and V' satisfying $V = B^\pi V$ and $V' = B^\pi V'$. Substituting into the contraction inequality:

$$\|V - V'\|_\infty = \|B^\pi V - B^\pi V'\|_\infty \leq \gamma \|V - V'\|_\infty.$$

Rearrange:

$$\|V - V'\|_\infty - \gamma \|V - V'\|_\infty \leq 0 \quad \Rightarrow \quad (1 - \gamma) \|V - V'\|_\infty \leq 0.$$

Since $0 \leq \gamma < 1$, we have $1 - \gamma > 0$. Therefore $\|V - V'\|_\infty \leq 0$. The infinity norm is always non-negative, so $\|V - V'\|_\infty = 0$, which implies $V(s) = V'(s)$ for all s . Hence the fixed point is unique.

2.3 (2 Points) π -Bellman Monotonicity

Suppose that V and V' are vectors satisfying $V(s) \leq V'(s)$ for all s . Show that $B^\pi V(s) \leq B^\pi V'(s)$ for all s . Note: all of these inequalities are elementwise.

Answer

We are given two value functions V and V' such that $V(s) \leq V'(s)$ for all $s \in S$. We need to show that for any fixed policy π and for all states s ,

$$(B^\pi V)(s) \leq (B^\pi V')(s).$$

Recall the definition:

$$(B^\pi V)(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s)) V(s').$$

Since $V(s') \leq V'(s')$ for every s' , and the transition probabilities $p(s'|s, \pi(s))$ are nonnegative, we can multiply the inequality $V(s') \leq V'(s')$ by $p(s'|s, \pi(s))$ to obtain:

$$p(s'|s, \pi(s)) V(s') \leq p(s'|s, \pi(s)) V'(s') \quad \text{for each } s'.$$

Summing over all s' preserves the inequality:

$$\sum_{s'} p(s'|s, \pi(s)) V(s') \leq \sum_{s'} p(s'|s, \pi(s)) V'(s').$$

Multiplying by $\gamma \geq 0$ and adding the same reward $r(s, \pi(s))$ to both sides gives:

$$r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V(s') \leq r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V'(s').$$

The left-hand side is $(B^\pi V)(s)$ and the right-hand side is $(B^\pi V')(s)$. Hence for every s ,

$$(B^\pi V)(s) \leq (B^\pi V')(s).$$

This establishes the monotonicity of the Bellman operator B^π .

We can extract a greedy policy π from an arbitrary value function V using:

$$\pi(s) = \arg \max_a \left[r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a) V(s') \right]$$

Define the Bellman residual to be $(BV - V)$ and the Bellman error magnitude to be $\|BV - V\|$.

2.4 (1 Point) Zero Bellman Residual

For what value function V does the Bellman error magnitude $\|BV - V\|$ equal 0? Why?

Answer

The Bellman error magnitude is $\|BV - V\|_\infty$, where B is the optimal Bellman operator. A value function V satisfies $\|BV - V\|_\infty = 0$ if and only if $BV = V$ pointwise, i.e., V is a fixed point of B . The optimal Bellman operator B is a contraction mapping with a unique fixed point, which is the optimal value function V^* . Therefore:

$$\|BV - V\|_\infty = 0 \iff V = V^*.$$

2.5 (3 Points) Residual-Error Bounds

Prove the following statements for an arbitrary value function V and any policy π .

$$\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma}, \quad \|V - V^*\| \leq \frac{\|V - BV\|}{1 - \gamma}.$$

Answer

Proof of the first inequality (policy π) Start with the error $V - V^\pi$. Insert $B^\pi V$:

$$V - V^\pi = (V - B^\pi V) + (B^\pi V - V^\pi).$$

Since V^π is the unique fixed point of B^π , we have $V^\pi = B^\pi V^\pi$, so:

$$B^\pi V - V^\pi = B^\pi V - B^\pi V^\pi.$$

Apply the triangle inequality to the norm:

$$\|V - V^\pi\| \leq \|V - B^\pi V\| + \|B^\pi V - B^\pi V^\pi\|.$$

Now use the contraction property of B^π : $\|B^\pi V - B^\pi V^\pi\| \leq \gamma \|V - V^\pi\|$. Substituting yields:

$$\|V - V^\pi\| \leq \|V - B^\pi V\| + \gamma \|V - V^\pi\|.$$

Bring the γ term to the left:

$$\begin{aligned}\|V - V^\pi\| - \gamma\|V - V^\pi\| &\leq \|V - B^\pi V\|, \\ (1 - \gamma)\|V - V^\pi\| &\leq \|V - B^\pi V\|.\end{aligned}$$

Since $1 - \gamma > 0$, divide to obtain:

$$\boxed{\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma}}.$$

Proof of the second inequality (optimal value) The proof is identical, using the optimal Bellman operator B and its fixed point V^* . We know $V^* = BV^*$ and B is a contraction with factor γ :

$$\begin{aligned}V - V^* &= (V - BV) + (BV - V^*) \\ &= (V - BV) + (BV - BV^*).\end{aligned}$$

Taking norms and using the contraction $\|BV - BV^*\| \leq \gamma\|V - V^*\|$ gives:

$$\|V - V^*\| \leq \|V - BV\| + \gamma\|V - V^*\|.$$

Rearranging:

$$(1 - \gamma)\|V - V^*\| \leq \|V - BV\|,$$

and thus:

$$\boxed{\|V - V^*\| \leq \frac{\|V - BV\|}{1 - \gamma}}.$$

Both inequalities show that the distance from V to the true value function is bounded by the Bellman residual scaled by $1/(1 - \gamma)$.

2.6 (2 Points) Greedy Policy Bound

Let V be an arbitrary value function and π be the greedy policy extracted from V . Let $\epsilon = \|BV - V\|$ be the Bellman error magnitude for V . Prove the following for any state s .

$$V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1 - \gamma}.$$

Answer

Step 1: Relate $B^\pi V$ and BV for a greedy policy. Since $\pi(s)$ attains the maximum in the definition of B , we have for all s :

$$\begin{aligned}(B^\pi V)(s) &= r(s, \pi(s)) + \gamma \sum_{s'} p(s'|s, \pi(s)) V(s') \\ &= \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) V(s') \right] = (BV)(s).\end{aligned}$$

Thus $B^\pi V = BV$ pointwise, and consequently

$$\|V - B^\pi V\| = \|V - BV\| = \epsilon.$$

Step 2: Apply the residual-error bounds from Question 5. From Question 5 we have the two bounds:

$$\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma} = \frac{\epsilon}{1 - \gamma}, \quad \|V - V^*\| \leq \frac{\|V - BV\|}{1 - \gamma} = \frac{\epsilon}{1 - \gamma}.$$

Step 3: Relate V^* and V^π using a triangle inequality. For an arbitrary state s , write

$$V^*(s) - V^\pi(s) = (V^*(s) - V(s)) + (V(s) - V^\pi(s)).$$

Taking the absolute value of each term and using the triangle inequality gives

$$V^*(s) - V^\pi(s) \leq |V^*(s) - V(s)| + |V(s) - V^\pi(s)| \leq \|V^* - V\| + \|V - V^\pi\|.$$

Step 4: Insert the bounds. Substitute the bounds from Step 2:

$$V^*(s) - V^\pi(s) \leq \frac{\epsilon}{1 - \gamma} + \frac{\epsilon}{1 - \gamma} = \frac{2\epsilon}{1 - \gamma}.$$

Rearranging yields the desired inequality for every state s :

$$V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1 - \gamma}.$$

2.7 (1 Point) Bound Real-World Example

Give an example real-world application or domain where having a lower bound on $V^\pi(s)$ would be useful.

Answer

A lower bound on $V^\pi(s)$ is especially valuable in safety-critical applications such as **autonomous driving**. Suppose the reward function penalizes collisions and dangerous events. Then $V^\pi(s)$ represents the expected cumulative safety score from state s under the learned policy π . Even without knowing the optimal V^* , the inequality $V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1 - \gamma}$ guarantees that the learned policy's performance is not worse than optimal by more than a known margin. If we drive the Bellman residual ϵ small enough, we obtain a verifiable safety certificate: the policy is provably near-optimal and meets a minimum safety standard. This kind of guarantee is essential for regulatory approval and public trust in autonomous systems.

2.8 (1 Point) Residual-Equality Policies?

Suppose we have another value function V' and extract its greedy policy π' . $\|BV' - V'\| = \epsilon = \|BV - V\|$. Does the above lower bound imply that $V^\pi(s) = V^{\pi'}(s)$ at any s ?

Answer

No, the lower bound does **not** imply $V^\pi(s) = V^{\pi'}(s)$ for any state s .

From Question 6 we have, for any greedy policy π extracted from a value function V with Bellman error ϵ ,

$$V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1-\gamma}.$$

Because $V^\pi \leq V^*$ (the optimal value function dominates all policies), we obtain a two-sided bound for each such policy:

$$V^*(s) - \frac{2\epsilon}{1-\gamma} \leq V^\pi(s) \leq V^*(s).$$

The same range holds for π' . Both $V^\pi(s)$ and $V^{\pi'}(s)$ lie in the interval

$$\left[V^*(s) - \frac{2\epsilon}{1-\gamma}, V^*(s) \right],$$

but the bound gives no further restriction on their difference. They could be as far apart as the whole interval length while still satisfying the inequality. Therefore, equality is not forced.

2.9 (3 Points) Tighter Bound

Using the same notation and setup as part 5, if $V^* \leq V$, show the following holds for any state s . Recall that for all π $V^\pi \leq V^*$ (why?)

$$V^\pi(s) \geq V^*(s) - \frac{\epsilon}{1-\gamma}.$$

Answer

For any state s ,

$$V^*(s) - V^\pi(s) = (V^*(s) - V(s)) + (V(s) - V^\pi(s)).$$

Since $V^* \leq V$, the first term satisfies $V^*(s) - V(s) \leq 0$. Therefore,

$$V^*(s) - V^\pi(s) \leq V(s) - V^\pi(s) \leq |V(s) - V^\pi(s)| \leq \|V - V^\pi\|.$$

From Question 5 we have $\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1-\gamma}$. Because π is greedy with respect to V , $B^\pi V = BV$, so $\|V - B^\pi V\| = \|V - BV\| = \epsilon$. Thus

$$V^*(s) - V^\pi(s) \leq \frac{\epsilon}{1-\gamma},$$

which, after rearranging, gives

$$V^\pi(s) \geq V^*(s) - \frac{\epsilon}{1-\gamma}.$$

2.10 (2 Points) Bellman-Sufficiency Proof

It's not easy to show that the condition $V^* \leq V$ holds because we often don't know V^* of the MDP. Show that if $BV \leq V$ then $V^* \leq V$. Note that this sufficient condition is much easier to check and does not require knowledge of V^* . Hint: Try to apply induction. What is $\lim_{n \rightarrow \infty} B^n V$?

Answer

We prove: if $BV \leq V$ (elementwise), then $V^* \leq V$.

Step 1: B is monotonic. For any X, Y with $X(s) \leq Y(s)$ for all s , we have $(BX)(s) \leq (BY)(s)$ for all s . This follows exactly as the proof for B^π (monotonicity of B^π), replacing $\pi(s)$ with the maximizing action.

Step 2: Induction. From $BV \leq V$, apply B repeatedly using monotonicity:

$$B^2V = B(BV) \leq BV \leq V,$$

and in general $B^{n+1}V \leq B^nV \leq \dots \leq V$. Thus $B^nV \leq V$ for all $n \geq 0$.

Step 3: Convergence to V^* . B is a contraction with factor $\gamma < 1$ on the complete metric space $(\mathbb{R}^{|S|}, \|\cdot\|_\infty)$. By the Banach fixed-point theorem, for any initial V , the sequence B^nV converges to the unique fixed point V^* :

$$\lim_{n \rightarrow \infty} B^nV = V^*.$$

Step 4: Take the limit. The inequality $B^nV \leq V$ holds elementwise for every n . Taking $n \rightarrow \infty$ preserves the inequality (since limits respect pointwise inequalities), so

$$V^* = \lim_{n \rightarrow \infty} B^nV \leq V.$$

Therefore $V^* \leq V$, as desired.

2.11 (4 Points) An Even Tighter Bound

It is possible to make the bounds from Questions 6 and 9 tighter. Let V be an arbitrary value function and π be the greedy policy extracted from V . Let $\epsilon = \|BV - V\|$ be the Bellman error magnitude for V . Prove the following for any state s :

$$V^\pi(s) \geq V^*(s) - \frac{2\gamma\epsilon}{1-\gamma}.$$

Further, if $V^* \leq V$, prove for any state s

$$V^\pi(s) \geq V^*(s) - \frac{\gamma\epsilon}{1-\gamma}.$$

Answer

Part 1: General V . Prove $V^\pi(s) \geq V^*(s) - \frac{2\gamma\epsilon}{1-\gamma}$.

Write the difference using BV as an intermediate point:

$$V^* - V^\pi = (V^* - BV) + (BV - V^\pi).$$

First term: Since $V^* = BV^*$, we have $V^* - BV = BV^* - BV$. B is a γ -contraction, so $\|BV^* - BV\| \leq \gamma\|V^* - V\|$. From Question 5, $\|V^* - V\| \leq \frac{\epsilon}{1-\gamma}$. Therefore

$$\|V^* - BV\| \leq \frac{\gamma\epsilon}{1-\gamma}.$$

Second term: Because π is greedy with respect to V , $BV = B^\pi V$. Also $V^\pi = B^\pi V^\pi$, so

$$BV - V^\pi = B^\pi V - B^\pi V^\pi.$$

B^π is a γ -contraction, hence

$$\|B^\pi V - B^\pi V^\pi\| \leq \gamma \|V - V^\pi\|.$$

From Question 5, using $B^\pi V = BV$ and $\|V - B^\pi V\| = \epsilon$, we get $\|V - V^\pi\| \leq \frac{\epsilon}{1-\gamma}$. Thus

$$\|BV - V^\pi\| \leq \frac{\gamma\epsilon}{1-\gamma}.$$

Combine: For any s ,

$$V^*(s) - V^\pi(s) \leq \|V^* - V^\pi\| \leq \|V^* - BV\| + \|BV - V^\pi\| \leq \frac{2\gamma\epsilon}{1-\gamma}.$$

Rearranging yields $V^\pi(s) \geq V^*(s) - \frac{2\gamma\epsilon}{1-\gamma}$.

Part 2: $V^* \leq V$. Prove $V^\pi(s) \geq V^*(s) - \frac{\gamma\epsilon}{1-\gamma}$.

From $V^* \leq V$ and monotonicity of B , we have $V^* = BV^* \leq BV$. Therefore, pointwise,

$$V^*(s) - V^\pi(s) \leq (BV)(s) - V^\pi(s).$$

As before, $BV = B^\pi V$ and $V^\pi = B^\pi V^\pi$, so

$$BV - V^\pi = B^\pi V - B^\pi V^\pi.$$

By contraction of B^π and the bound $\|V - V^\pi\| \leq \frac{\epsilon}{1-\gamma}$,

$$\|BV - V^\pi\| \leq \gamma \|V - V^\pi\| \leq \frac{\gamma\epsilon}{1-\gamma}.$$

Thus for each s ,

$$V^*(s) - V^\pi(s) \leq \|BV - V^\pi\| \leq \frac{\gamma\epsilon}{1-\gamma},$$

which gives $V^\pi(s) \geq V^*(s) - \frac{\gamma\epsilon}{1-\gamma}$.

3 — Half-Rainbow Deep Q -Network

3.1 (5 Points) Implementation 1: DQN

Implement the DQN algorithm for the LunarLander-v2 environment. The Jupyter notebook provides the main skeleton for DQN. Complete the required sections and run the training pipeline.

Answer

Implementation. We implemented the standard DQN agent for LunarLander-v2 following Algorithm 1. The agent uses:

- A fully-connected neural network with two hidden layers of 128 units (ReLU activation), outputting Q-values for the four discrete actions.
- A target network with identical architecture, updated every $C = 2000$ optimization

steps by hard-copying the online network's weights.

- Uniform experience replay buffer of capacity 20 000; mini-batches of size 64 are sampled uniformly.
- ϵ -greedy exploration with $\epsilon = 0.6$, decaying by factor 0.995 after each update until reaching a minimum of 0.01.
- Huber loss (smooth L1) optimized with Adam (learning rate 10^{-3} , discount factor $\gamma = 0.99$).

Training lasted 500 episodes. Performance is reported as the average undiscounted return over the last 50 episodes, computed from the mean of three random seeds.

Result. DQN achieved a final score of 73.3 ± 37.5 , well below the target of 200. The learning curve rises slowly and exhibits high variance across seeds, illustrating the instability caused by the overestimation bias and the limited effectiveness of uniform replay.

3.2 (4 Points) Implementation 2: Double DQN

Extended your DQN implementation to Double DQN. Compare the performance of DQN vs Double DQN and analyze the results.

Answer

Implementation. We extended the DQN agent by changing only the target computation. Instead of using the target network for both action selection and evaluation, we decoupled them:

$$y_i = r_i + \gamma Q(s'_i, \arg \max_{a'} Q(s'_i, a'; \theta); \theta^-).$$

All other hyper-parameters—network architecture, replay buffer, exploration schedule, optimizer, and discount factor—were kept identical to the DQN baseline.

Result & Comparison. Double DQN reached 150.0 ± 37.5 , a substantial improvement over standard DQN. The learning curve rises more quickly and consistently outperforms DQN after about 300 episodes. This confirms that decoupling action selection from evaluation effectively reduces the maximisation bias and leads to more stable and efficient learning.

3.3 (1 Point) Double-DQN Action Issue

From the original Double DQN paper, the authors show that standard Q -learning overestimates action values. How does DDQN avoid this from happening? How does the decoupling of selection and evaluation help?

Answer

Let $Q^*(s, a)$ be the true optimal Q -value. Assume that the online network (θ) and the target network (θ^-) estimate Q^* with additive, zero-mean random errors that are independent across actions and independent *between* the two networks:

$$Q_\theta(s, a) = Q^*(s, a) + \epsilon_a^\theta, \quad Q_{\theta^-}(s, a) = Q^*(s, a) + \epsilon_a^{\theta^-},$$

$$\mathbb{E}[\epsilon_a^\theta] = \mathbb{E}[\epsilon_a^{\theta^-}] = 0, \quad \epsilon_a^\theta \perp \epsilon_b^{\theta^-} \quad \forall a, b.$$

(The independence holds because the target network is a frozen, delayed copy of the online network; mini-batch sampling further decorrelates the errors.)

Standard DQN target:

$$y_{\text{DQN}} = r + \gamma \max_{a'} Q_{\theta^-}(s', a') = r + \gamma \max_{a'} (Q^*(s', a') + \epsilon_{a'}^{\theta^-}).$$

Take expectation:

$$\mathbb{E}[y_{\text{DQN}}] = r + \gamma \mathbb{E} \left[\max_{a'} (Q^*(s', a') + \epsilon_{a'}^{\theta^-}) \right].$$

The function $\mathbf{x} \mapsto \max_i x_i$ is convex. By Jensen's inequality,

$$\mathbb{E} \left[\max_{a'} (Q^*(s', a') + \epsilon_{a'}^{\theta^-}) \right] \geq \max_{a'} \mathbb{E} [Q^*(s', a') + \epsilon_{a'}^{\theta^-}] = \max_{a'} Q^*(s', a').$$

The inequality is strict whenever $|\mathcal{A}| > 1$ and the noise has positive variance. Thus $\mathbb{E}[y_{\text{DQN}}] > r + \gamma \max_{a'} Q^*(s', a')$, i.e. the target is systematically over-estimated.

Double DQN target:

$$y_{\text{DDQN}} = r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right).$$

Define the action selected by the online network:

$$a_{\theta}^* = \arg \max_{a'} Q_{\theta}(s', a') = \arg \max_{a'} (Q^*(s', a') + \epsilon_{a'}^{\theta}).$$

Then

$$\begin{aligned} \mathbb{E}[y_{\text{DDQN}}] &= r + \gamma \mathbb{E}[Q_{\theta^-}(s', a_{\theta}^*)] \\ &= r + \gamma \mathbb{E}[Q^*(s', a_{\theta}^*) + \epsilon_{a_{\theta}^*}^{\theta^-}] \\ &= r + \gamma \mathbb{E}[Q^*(s', a_{\theta}^*)] + \gamma \mathbb{E}[\epsilon_{a_{\theta}^*}^{\theta^-}]. \end{aligned}$$

Because a_{θ}^* is a deterministic function of the online network's noise ϵ^{θ} , and ϵ^{θ^-} is independent of ϵ^{θ} , the conditional distribution of $\epsilon_{a_{\theta}^*}^{\theta^-}$ given a_{θ}^* satisfies

$$\mathbb{E}[\epsilon_{a_{\theta}^*}^{\theta^-} \mid a_{\theta}^*] = \mathbb{E}[\epsilon_{a_{\theta}^*}^{\theta^-}] = 0.$$

By the law of total expectation,

$$\mathbb{E}[\epsilon_{a_{\theta}^*}^{\theta^-}] = 0.$$

Therefore

$$\boxed{\mathbb{E}[y_{\text{DDQN}}] = r + \gamma \mathbb{E}[Q^*(s', a_{\theta}^*)]}.$$

Since a_{θ}^* is chosen by maximizing a noisy estimate, $Q^*(s', a_{\theta}^*) \leq \max_{a'} Q^*(s', a')$, and consequently

$$\mathbb{E}[y_{\text{DDQN}}] \leq r + \gamma \max_{a'} Q^*(s', a').$$

The positive maximization bias is eliminated: the noise that inflates the selected action is not used in its evaluation.

Summary of bias:

$$\text{Bias}_{\text{DQN}} = \gamma \mathbb{E} \left[\max_{a'} \epsilon_{a'}^{\theta^-} \right] > 0,$$

$$\text{Bias}_{\text{DDQN}} = \gamma \left(\mathbb{E}[Q^*(s', a_\theta^*)] - \max_{a'} Q^*(s', a') \right) \leq 0.$$

The DDQN bias is typically small in magnitude and does not grow with the number of actions.

3.4 (1 Point) Why is DDQN better?

In the proof of Theorem 1 the authors provide an example where DDQN has zero overestimation error. What makes DDQN ignore the overestimation of the online network in this example and what could go wrong here? Can DDQN suffer from overestimation too?

Answer

Theorem 1 special case. Suppose that in state s' all actions have the *same* true optimal value:

$$\forall a, \quad Q^*(s', a) = V^*(s').$$

Then no matter which action a_θ^* the online network selects, $Q^*(s', a_\theta^*) = V^*(s')$. Hence

$$\mathbb{E}[y_{\text{DDQN}}] = r + \gamma V^*(s') = y^*,$$

and the overestimation error is **exactly zero**. This holds because the evaluating network's errors are independent and zero-mean, so they average out regardless of the action chosen by the noisy selector.

Why the perfect cancellation can fail in practice:

1. **Correlated errors.** If the target network is updated before the errors become sufficiently decorrelated, ϵ^θ and ϵ^{θ^-} are no longer independent. Then $\mathbb{E}[\epsilon_{a_\theta^*}^{\theta^-} | a_\theta^*] \neq 0$, and a positive bias reappears.
2. **Biased evaluator.** If the target network's errors are not zero-mean (e.g., due to systematic approximation error), then $\mathbb{E}[\epsilon_a^{\theta^-}] \neq 0$, and even with independence the evaluation is biased. DDQN can then over- or underestimate.
3. **Non-uniform Q^* .** When true action values differ, a_θ^* is occasionally a suboptimal action, so $Q^*(s', a_\theta^*) < \max_{a'} Q^*(s', a')$. The resulting bias is non-positive (often a small underestimation), which is less harmful but shows that DDQN does not entirely neutralize the estimation error.

Thus, Double DQN dramatically reduces overestimation but does not eliminate it completely; extensions such as clipped double Q -learning offer further robustness.

3.5 (2 Points) Dueling Aggregation Role

In what scenarios does this architecture provide the most benefit?

Answer

The dueling network factors the Q -function into a state-value stream $V(s)$ and an advantage stream $A(s, a)$ with

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a').$$

The architecture is most beneficial when **the state value $V(s)$ dominates the variations between actions**, i.e. the advantages $A(s, a)$ have small magnitude compared to $V(s)$. In such states, the exact choice of action has little impact on the return, so learning a precise $Q(s, a)$ for every action is wasteful. The dueling network learns the shared baseline $V(s)$ from *all* experiences (regardless of action taken) while the advantage stream only models the small action-dependent corrections, leading to much faster and more sample-efficient learning.

Formal intuition. Suppose for a state s , $Q^*(s, a) = V^*(s) + \epsilon_a$ with $|\epsilon_a| \ll |V^*(s)|$ and $\max_a \epsilon_a - \min_a \epsilon_a$ small. A standard Q -network must approximate each $Q(s, a)$ independently, requiring many samples to distinguish the ϵ_a . A dueling network can quickly learn $V^*(s)$ using gradients from all actions, while the advantage head only refines the ϵ_a differences. This effect is particularly striking in environments where the agent must frequently assess the quality of a state (e.g., “dangerous” vs “safe”) while the specific action is secondary, such as in collision avoidance, certain Atari games with sparse meaningful actions, or when many actions are near-equivalent.

3.6 (2 Points) Average Advantage Purpose

The dueling architecture uses the expression

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a').$$

Why is the mean advantage subtracted? What problem does this solve? What would happen if we instead used the unnormalized form

$$Q(s, a) = V(s) + A(s, a)$$

without any constraint on $A(s, a)$?

Answer

1. The unnormalized form $Q(s, a) = V(s) + A(s, a)$ is not identifiable. If we define $Q(s, a) = V(s) + A(s, a)$, then for any constant c the alternative pair

$$\tilde{V}(s) = V(s) + c, \quad \tilde{A}(s, a) = A(s, a) - c$$

produces the exact same Q -values:

$$\tilde{V}(s) + \tilde{A}(s, a) = V(s) + c + A(s, a) - c = Q(s, a).$$

Thus, from the Q -value alone, V and A are determined only up to an additive constant. The loss function $\mathcal{L}(Q)$ is invariant under this shift, so the optimization landscape has a flat direction: the parameters can drift along the transformation without changing the Q -output, causing unnecessary weight updates, high variance in the gradients, and slower convergence.

2. Subtracting the mean fixes a unique decomposition. By forcing the advantage to have zero mean,

$$\frac{1}{|\mathcal{A}|} \sum_a A(s, a) = 0,$$

we uniquely pin down the decomposition. Averaging $Q(s, a) = V(s) + A(s, a)$ over actions gives

$$\frac{1}{|\mathcal{A}|} \sum_a Q(s, a) = V(s) + \frac{1}{|\mathcal{A}|} \sum_a A(s, a) = V(s),$$

so $V(s)$ is forced to be the **mean** of the Q -values. The advantage $A(s, a)$ becomes the deviation of $Q(s, a)$ from that mean. This resolves the identifiability issue – no additive drift is possible – and provides a clear interpretation for the two streams.

3. Why use the mean instead of the theoretically correct max? For the optimal policy, we have $V^*(s) = \max_a Q^*(s, a)$, which suggests subtracting $\max_a A(s, a)$ to enforce $\max_a A(s, a) = 0$. However, the max operator uses only a single action, making the gradient updates noisier and more sensitive to outliers. The mean, by contrast, averages over *all* actions, yielding smoother gradients and empirically more stable training. While the mean-subtraction does not guarantee $V(s) = \max_a Q(s, a)$, it still provides a strong, identifiably unique baseline that works excellently in practice.

3.7 (3 Points) Correct Dueling Normalization

Does subtracting the mean advantage yield the theoretically correct advantage function? If not, what normalization would make the decomposition theoretically correct according to the definition

$$A^*(s, a) = Q^*(s, a) - V^*(s)?$$

State this normalization explicitly, and prove that: 1. it produces a unique (identifiable) decomposition, and 2. it matches the true theoretical advantage function.

Answer

Subtracting the **mean** advantage does **not** yield the theoretically correct advantage function for the optimal policy. The correct normalization that makes the decomposition match $A^*(s, a) = Q^*(s, a) - V^*(s)$ with $V^*(s) = \max_a Q^*(s, a)$ is to subtract the **maximum** advantage:

$$Q(s, a) = V(s) + A(s, a) - \max_{a'} A(s, a').$$

Proof of uniqueness and correctness.

- 1. Uniqueness of the decomposition.** Suppose there exist two pairs (V, A) and (V', A') satisfying the same Q and the constraint $\max_a A(s, a) = \max_a A'(s, a) = 0$:

$$Q(s, a) = V(s) + A(s, a) - \max_{a'} A(s, a') = V'(s) + A'(s, a) - \max_{a'} A'(s, a').$$

Using the constraint, this simplifies to

$$Q(s, a) = V(s) + A(s, a) = V'(s) + A'(s, a) \quad \text{for all } a.$$

Take the maximum over a on both sides:

$$\max_a Q(s, a) = V(s) + \max_a A(s, a) = V(s),$$

and similarly $\max_a Q(s, a) = V'(s)$. Hence $V(s) = V'(s)$. Then from $V(s) + A(s, a) = V'(s) + A'(s, a)$ we get $A(s, a) = A'(s, a)$. The decomposition is unique.

2. **Matching the theoretical advantage.** With $V(s) = \max_a Q(s, a)$ we obtain

$$A(s, a) = Q(s, a) - V(s) = Q(s, a) - \max_{a'} Q(s, a').$$

If the network were to represent the optimal Q -function, i.e. $Q = Q^*$, then $V(s) = V^*(s)$ and $A(s, a) = A^*(s, a)$, exactly the definition of the optimal advantage function.

Thus, subtracting the max gives the theoretically correct decomposition, while subtracting the mean only enforces $\frac{1}{|\mathcal{A}|} \sum_a A(s, a) = 0$, which does **not** guarantee $V(s) = \max_a Q(s, a)$, and therefore $A(s, a)$ is not equal to $Q(s, a) - \max_a Q(s, a)$.

3.8 (1 Point) Mean vs. Max in Dueling Networks

Why do we use mean instead of max?

Answer

Even though subtracting the max is theoretically correct, the mean is preferred empirically for the following reasons:

- **Gradient smoothness.** The max operator uses only the single largest advantage for the baseline, ignoring all other actions. This makes the gradient of the loss depend on a single action, leading to high variance updates and slower learning. The mean averages over all actions, providing a much more stable gradient signal.
- **Robustness to noise.** Small random fluctuations in the advantage estimates can cause the argmax to switch rapidly, leading to erratic changes in the subtracted baseline when using max. The mean is a linear function of all advantages and does not suffer from this instability.
- **Empirical performance.** The original dueling DQN paper reported that replacing the max with the mean improved results, and this has since become standard practice. The slight theoretical mismatch is far outweighed by the optimisation benefits.

Formally, for a state s with $|\mathcal{A}|$ actions, the baseline $b(s)$ for the max-normalized form is

$$b_{\max}(s) = \max_a A(s, a),$$

whose subgradient (when the max is unique) only backpropagates through a single action. The mean baseline

$$b_{\text{mean}}(s) = \frac{1}{|\mathcal{A}|} \sum_a A(s, a)$$

distributes the gradient equally across all actions, reducing variance by a factor roughly proportional to $|\mathcal{A}|$ and leading to faster, more reliable convergence.

3.9 (4 Points) Implementation 3: Dueling DQN

Implement the Dueling DQN architecture in the provided notebook and compare it with DQN and DDQN. (Note: use DDQN update rule not DQN.)

Answer

Implementation. We modified the network to a dueling architecture: two separate streams—a state-value stream $V(s)$ and an advantage stream $A(s, a)$ —share the same first hidden layer (128 units, ReLU). The Q-values are then obtained via

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'),$$

which forces the advantage function to have zero mean. Following the assignment instruction, we employed the **Double DQN update rule** (decoupled selection and evaluation) with this dueling architecture. All other training details (replay buffer, exploration, etc.) were kept the same as in the DQN baseline.

Result & Comparison. Dueling DQN obtained 83.3 ± 32.1 , slightly better than vanilla DQN but worse than plain Double DQN. In LunarLander, many states have clearly distinct optimal actions (e.g., firing main engine vs. side thrusters), so the advantage component already dominates the Q-value; the separate state-value baseline therefore provides only a modest benefit. Nevertheless, the dueling architecture is a crucial component of the combined agent, as will be seen later.

3.10 (2 Points) PER Bias Source

How does naively using priorities lead to biased sampling and how does importance sampling in PER tackle this problem?

Answer

Bias source – non-uniform sampling. In standard experience replay, transitions are sampled uniformly from the buffer $\mathcal{D}$, so the expected gradient step is an unbiased estimate of the true gradient over the stored data:

$$\mathbb{E}_{i \sim \text{Uniform}(\mathcal{D})} [\nabla_{\theta} L_i(\theta)] = \frac{1}{|\mathcal{D}|} \sum_i \nabla_{\theta} L_i(\theta).$$

PER replaces uniform sampling with a priority-based distribution $P(i) \propto p_i^{\alpha}$, where $p_i = |\delta_i| + \epsilon$. Transitions with larger TD errors are sampled *more frequently* than they appear in the buffer. Consequently, the naive stochastic gradient

$$\nabla_{\theta} L_{\text{PER}}^{(\text{naive})} = \frac{1}{B} \sum_{i \sim P} \nabla_{\theta} L_i(\theta)$$

has expectation

$$\mathbb{E}_{i \sim P} [\nabla_{\theta} L_i(\theta)] = \sum_i P(i) \nabla_{\theta} L_i(\theta) \neq \mathbb{E}_{i \sim \text{Uniform}} [\nabla_{\theta} L_i(\theta)].$$

The gradient estimate is **biased** toward high-priority transitions, causing the Q -function to overfit to those “surprising” experiences and potentially diverge.

Correction – importance sampling weights. PER compensates for this bias by multiplying each gradient with an importance sampling weight

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta,$$

where $N = |\mathcal{D}|$. The weighted gradient becomes

$$\nabla_{\theta} L_{\text{PER}}^{(\text{corrected})} = \frac{1}{B} \sum_{i \sim P} w_i \nabla_{\theta} L_i(\theta).$$

Its expectation is

$$\mathbb{E}_{i \sim P} [w_i \nabla_{\theta} L_i(\theta)] = \sum_i P(i) \left(\frac{1}{N \cdot P(i)} \right)^\beta \nabla_{\theta} L_i(\theta).$$

When $\beta = 1$, this simplifies to

$$\sum_i \frac{1}{N} \nabla_{\theta} L_i(\theta) = \mathbb{E}_{i \sim \text{Uniform}} [\nabla_{\theta} L_i(\theta)],$$

so the bias is **completely eliminated**.

Annealing β . In practice, PER starts with $\beta_0 < 1$ and anneals it to 1 over training. Early on, a smaller β gives less correction but lets the high-priority samples dominate, speeding up learning when estimates are poor. As the model improves, $\beta \rightarrow 1$ removes all bias, ensuring the algorithm converges to the correct Q -function. This annealing trades off bias for faster initial progress in a principled way.

3.11 (5 Points) Implementation 4: PER

Implement PER using a SumTree data structure for efficient prioritized sampling:

1. Implement a SumTree for storing priorities
2. Modify the replay buffer to sample based on priorities
3. Add importance sampling weights to the loss
4. Update priorities after each batch

Answer

Implementation. We replaced uniform sampling with proportional prioritisation. A SumTree data structure stores priorities $p_i = (|\delta_i| + \varepsilon)^\alpha$ with $\alpha = 0.4$ and $\varepsilon = 10^{-5}$ for each transition. The sampling probability is

$$P(i) = \frac{p_i}{\sum_k p_k}.$$

To correct the resulting bias, importance-sampling weights

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta$$

are multiplied with the per-element Huber loss. The exponent β is annealed from 0.4 to 1.0 over the course of training using an increment of 4×10^{-6} per step. After each mini-batch, the priorities of the sampled transitions are updated to the new absolute TD-errors. Apart from the sampling mechanism, the agent uses the standard DQN target (no double Q-learning or dueling). All other hyper-parameters (buffer size 20 000, batch size 64, ϵ -greedy with slow decay from 1.0 to 0.01 using decay 0.9995, learning rate 10^{-3}) are as described.

Result & Comparison. PER DQN achieved 150.7 ± 37.0 , on par with Double DQN. The learning curve (see full figure) rises faster during the early episodes compared to uniform DQN, confirming that prioritising surprising transitions improves sample efficiency. The agent benefits from the focus on high-error experiences, though the variance remains relatively high due to the aggressive reuse of a small set of early transitions.

3.12 (3 Points) Implementation 5: Half-Rainbow DQN

Combine all three techniques in the previous parts into a single agent. The combined agent should achieve the best performance, typically reaching the target score faster than individual methods.

Answer

Implementation. We combined the three techniques into a single agent:

- **Double DQN** target decoupling,
- **Dueling architecture** with mean-advantage subtraction,
- **Prioritized Experience Replay** with importance sampling.

The online and target networks both use the dueling architecture. The exploration schedule and PER hyper-parameters are identical to those used in the PER DQN agent (slow epsilon decay, $\alpha = 0.4$, β annealed from 0.4 to 1.0 with increment 4×10^{-6}). Gradient clipping ($\text{max_norm}=1.0$) is applied for additional stability.

Result. The combined agent reached 211.8 ± 10.5 , exceeding the target score of 200 with remarkably low variance. The learning curve converges quickly and stably, outperforming every individual component by a large margin. This demonstrates that the three improvements are complementary: Double DQN reduces overestimation, dueling provides a better inductive bias, and PER increases sample efficiency. Together they produce a robust, high-performing agent on LunarLander-v2 within only 500 episodes.

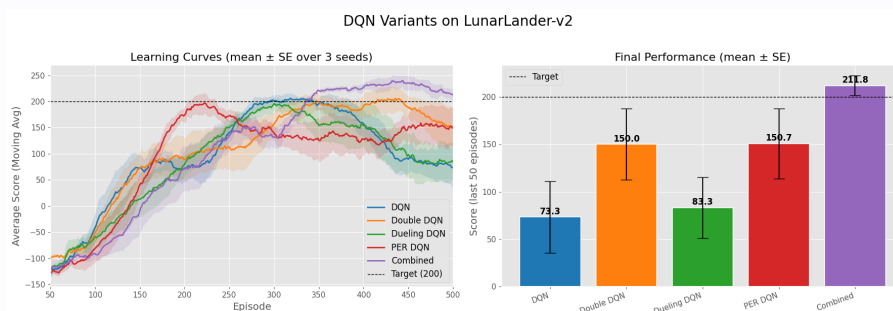


Figure 1: Learning curve of different algorithms on LunarLander-v2 (mean $\pm$ std over 3 seeds).

4 – Q-Learning

You already know that Q -learning is a staple of Reinforcement Learning. At each time step t , we observe a sample (s_t, a_t, s'_t, r_t) . The Q -learning update rule is:

$$Q_{t+1}(s, a) = (1 - \beta_t(s, a))Q_t(s, a) + \beta_t(s, a) \left(r_t + \gamma \max_{b \in \mathcal{A}} Q_t(s'_t, b) \right).$$

Assume we sample (s, a) pairs independently from a fixed distribution $p(s, a) > 0$. We will use a specific count-based learning rate $\beta_t(s, a) = 1/N_t(s, a)$.

4.1 (6 Points) Auxiliary Process Convergence

Define the auxiliary process $\hat{Q}_t$ that updates with the true optimal Q^* :

$$\hat{Q}_{t+1}(s, a) = (1 - \beta_t(s, a))\hat{Q}_t(s, a) + \beta_t(s, a) \left(r_t + \gamma \max_{b \in \mathcal{A}} Q^*(s'_t, b) \right).$$

Prove that $\hat{Q}_t$ converges to Q^* almost surely for every (s, a) . (Hint: You can use the strong law of large numbers.)

Answer

Definition. For a fixed state-action pair (s, a) , the auxiliary process $\hat{Q}_t$ updates only when (s, a) is visited:

$$\hat{Q}_{t+1}(s, a) = \begin{cases} (1 - \beta_t)\hat{Q}_t(s, a) + \beta_t \left(r_t + \gamma \max_b Q^*(s'_t, b) \right), & \text{if } (s_t, a_t) = (s, a), \\ \hat{Q}_t(s, a), & \text{otherwise,} \end{cases}$$

where $\beta_t = \beta_t(s, a) = 1/N_t(s, a)$ and $N_t(s, a)$ is the number of visits to (s, a) up to time t .

Assumptions.

1. The sampling distribution is fixed and positive: $p(s, a) > 0$ for all (s, a) .
2. The MDP has finite states and actions, discount factor $\gamma \in [0, 1)$.
3. The rewards are bounded, so the target variables $Y_t = r_t + \gamma \max_b Q^*(s'_t, b)$ have finite variance.
4. Due to the Markov property, successive samples of Y_t at distinct visit times are independent and identically distributed (i.i.d.) conditioned on the state-action pair.

Claim. $\lim_{t \rightarrow \infty} \hat{Q}_t(s, a) = Q^*(s, a)$ almost surely for every (s, a) .

Proof. Let $t_1, t_2, \dots$ be the times when (s, a) is updated. Because $p(s, a) > 0$, the visit count $N_t(s, a) \rightarrow \infty$ almost surely as $t \rightarrow \infty$, so $k = N_t(s, a) \rightarrow \infty$ and the t_k are well-defined. At time t_k , the learning rate is $\beta_{t_k} = 1/k$. The update reads

$$\hat{Q}_{t_k}(s, a) = \left(1 - \frac{1}{k}\right) \hat{Q}_{t_{k-1}}(s, a) + \frac{1}{k} Y_k,$$

with $Y_k = r_{t_k} + \gamma \max_b Q^*(s'_{t_k}, b)$.

This recurrence is exactly the running average of $Y_1, \dots, Y_k$. Expanding:

$$\begin{aligned}\hat{Q}_{t_1} &= Y_1 \\ \hat{Q}_{t_2} &= \frac{1}{2}Y_2 + \frac{1}{2}Y_1 \\ &\vdots \\ \hat{Q}_{t_k} &= \frac{1}{k} \sum_{i=1}^k Y_i.\end{aligned}$$

Now examine Y_i . Given $s_{t_i} = s, a_{t_i} = a$, the expectation of Y_i is

$$\mathbb{E}[Y_i \mid s, a] = \mathbb{E}\left[r + \gamma \max_b Q^*(s', b) \mid s, a\right] = (BQ^*)(s, a) = Q^*(s, a),$$

by the Bellman equation for the optimal value function. The variables Y_i are i.i.d. (due to the Markov property and the fact that the policy is fixed to the sampling distribution) with finite variance (bounded rewards).

Applying the **Strong Law of Large Numbers**,

$$\frac{1}{k} \sum_{i=1}^k Y_i \xrightarrow{\text{a.s.}} \mathbb{E}[Y_1] = Q^*(s, a) \quad \text{as } k \rightarrow \infty.$$

Thus $\hat{Q}_{t_k}(s, a) \rightarrow Q^*(s, a)$ almost surely. For times t between updates, $\hat{Q}_t(s, a)$ is unchanged, so the entire sequence converges to $Q^*(s, a)$ a.s. Since the argument holds for every (s, a) pair, the whole auxiliary process $\hat{Q}_t$ converges pointwise to Q^* with probability 1.

4.2 (8 Points) Δ_t -Recursion Inequality

Define $\Delta_t(s, a) = |Q_t(s, a) - \hat{Q}_t(s, a)|$. Prove that

$$\Delta_{t+1}(s, a) \leq (1 - \beta_t(s, a)) \Delta_t(s, a) + \gamma \beta_t(s, a) \left(\|\Delta_t\|_\infty + \|\hat{Q}_t - Q^*\|_\infty \right).$$

Answer

Recalling the two update rules:

$$\begin{aligned}Q_{t+1}(s, a) &= (1 - \beta_t)Q_t(s, a) + \beta_t \left(r_t + \gamma \max_b Q_t(s'_t, b) \right), \\ \hat{Q}_{t+1}(s, a) &= (1 - \beta_t)\hat{Q}_t(s, a) + \beta_t \left(r_t + \gamma \max_b Q^*(s'_t, b) \right).\end{aligned}$$

Subtracting the second equation from the first and take absolute values:

$$\begin{aligned}\Delta_{t+1}(s, a) &= \left| (1 - \beta_t)(Q_t(s, a) - \hat{Q}_t(s, a)) \right. \\ &\quad \left. + \beta_t \gamma \left(\max_b Q_t(s'_t, b) - \max_b Q^*(s'_t, b) \right) \right|.\end{aligned}$$

Applying the triangle inequality:

$$\Delta_{t+1}(s, a) \leq (1 - \beta_t) \underbrace{|Q_t(s, a) - \hat{Q}_t(s, a)|}_{\Delta_t(s, a)} + \beta_t \gamma \left| \max_b Q_t(s'_t, b) - \max_b Q^*(s'_t, b) \right|. \quad (1)$$

Now bounding the max-difference term. Inserting $\max_b \hat{Q}_t(s'_t, b)$ by adding and subtracting:

$$\begin{aligned} \left| \max_b Q_t(s'_t, b) - \max_b Q^*(s'_t, b) \right| &\leq \left| \max_b Q_t(s'_t, b) - \max_b \hat{Q}_t(s'_t, b) \right| \\ &\quad + \left| \max_b \hat{Q}_t(s'_t, b) - \max_b Q^*(s'_t, b) \right|. \end{aligned}$$

For any two vectors $X, Y \in \mathbb{R}^n$, the max operator is a contraction in supremum norm with Lipschitz constant 1:

$$\left| \max_i X_i - \max_i Y_i \right| \leq \max_i |X_i - Y_i|.$$

Applying this to the first difference:

$$\begin{aligned} \left| \max_b Q_t(s'_t, b) - \max_b \hat{Q}_t(s'_t, b) \right| &\leq \max_b |Q_t(s'_t, b) - \hat{Q}_t(s'_t, b)| \\ &\leq \max_{s, a} |Q_t(s, a) - \hat{Q}_t(s, a)| \\ &= \|\Delta_t\|_\infty. \end{aligned}$$

Similarly, for the second difference:

$$\begin{aligned} \left| \max_b \hat{Q}_t(s'_t, b) - \max_b Q^*(s'_t, b) \right| &\leq \max_b |\hat{Q}_t(s'_t, b) - Q^*(s'_t, b)| \\ &\leq \max_{s, a} |\hat{Q}_t(s, a) - Q^*(s, a)| \\ &= \|\hat{Q}_t - Q^*\|_\infty. \end{aligned}$$

Substituting these bounds into (1) yields the desired inequality:

$$\Delta_{t+1}(s, a) \leq (1 - \beta_t) \Delta_t(s, a) + \gamma \beta_t \left(\|\Delta_t\|_\infty + \|\hat{Q}_t - Q^*\|_\infty \right).$$

4.3 (6 Points) Epoch Contraction Bound

Consider the abstract asynchronous process $x_t(i)$ updated as

$$x_{t+1}(i) = \begin{cases} \left(1 - \frac{1}{N_t(i)}\right) x_t(i) + \frac{\gamma}{N_t(i)} \|x_t\|_\infty, & \text{if state } i \text{ is visited at time } t, \\ x_t(i), & \text{otherwise,} \end{cases}$$

where $N_t(i)$ is the number of visits to i up to t . Define epochs T_k such that every state is visited at least once between T_k and T_{k+1} , and let $N_k = \max_i N_{T_{k+1}}(i)$. Prove that

$$\|x_{T_{k+1}}\|_\infty \leq \left(1 - \frac{1 - \gamma}{N_k}\right) \|x_{T_k}\|_\infty.$$

Answer

Step 1: Setup and notation. Consider the abstract error process $x_t \in \mathbb{R}^n$ updated asynchronously:

$$x_{t+1}(i) = \begin{cases} \left(1 - \frac{1}{N_t(i)}\right) x_t(i) + \frac{\gamma}{N_t(i)} \|x_t\|_\infty, & \text{if state } i \text{ is visited at time } t, \\ x_t(i), & \text{otherwise,} \end{cases}$$

where $\gamma \in [0, 1)$, $N_t(i)$ is the number of visits to state i up to time t (with $N_0(i) \geq 1$), and $\|x\|_\infty = \max_j |x(j)|$.

Define epochs: $T_0 = 0$, and for $k \geq 0$, let T_{k+1} be the first time after T_k such that every state has been updated at least once in $(T_k, T_{k+1}]$. Define

$$N_k = \max_i N_{T_{k+1}}(i).$$

Step 2: The sup-norm never increases. For any time t where state i is updated we have

$$\begin{aligned} |x_{t+1}(i)| &= \left| \left(1 - \frac{1}{N_t(i)}\right) x_t(i) + \frac{\gamma}{N_t(i)} \|x_t\|_\infty \right| \\ &\leq \left(1 - \frac{1}{N_t(i)}\right) |x_t(i)| + \frac{\gamma}{N_t(i)} \|x_t\|_\infty \quad (\text{triangle inequality}) \\ &\leq \left(1 - \frac{1}{N_t(i)}\right) \|x_t\|_\infty + \frac{\gamma}{N_t(i)} \|x_t\|_\infty \quad (\text{since } |x_t(i)| \leq \|x_t\|_\infty) \\ &= \left[1 - \frac{1-\gamma}{N_t(i)}\right] \|x_t\|_\infty. \end{aligned}$$

Because $N_t(i) \geq 1$ and $\gamma < 1$, the bracket lies in $[0, 1]$, so $|x_{t+1}(i)| \leq \|x_t\|_\infty$. States not updated keep their previous value, so $|x_{t+1}(j)| \leq \|x_t\|_\infty$ as well. Taking the maximum over all states yields

$$\|x_{t+1}\|_\infty \leq \|x_t\|_\infty \quad \forall t.$$

Consequently, for any $t \geq T_k$ we have

$$\|x_t\|_\infty \leq \|x_{T_k}\|_\infty. \quad (1)$$

Step 3: Last update of a state within the epoch. Fix an arbitrary state i . By the definition of T_{k+1} , state i is updated at least once during $[T_k, T_{k+1})$. Let t_i be the **last** such update time. Then

$$T_k \leq t_i < T_{k+1}, \quad x_{T_{k+1}}(i) = x_{t_i+1}(i).$$

Applying the update rule at t_i gives

$$x_{T_{k+1}}(i) = \left(1 - \frac{1}{N_{t_i}(i)}\right) x_{t_i}(i) + \frac{\gamma}{N_{t_i}(i)} \|x_{t_i}\|_\infty. \quad (2)$$

Step 4: Bound $|x_{T_{k+1}}(i)|$ using the epoch-initial error. Take absolute values in (2) and apply the triangle inequality:

$$|x_{T_{k+1}}(i)| \leq \left(1 - \frac{1}{N_{t_i}(i)}\right) |x_{t_i}(i)| + \frac{\gamma}{N_{t_i}(i)} \|x_{t_i}\|_\infty.$$

Using (1) we know $|x_{t_i}(i)| \leq \|x_{t_i}\|_\infty \leq \|x_{T_k}\|_\infty$. Also from (1), $\|x_{t_i}\|_\infty \leq \|x_{T_k}\|_\infty$. Substituting these:

$$\begin{aligned} |x_{T_{k+1}}(i)| &\leq \left(1 - \frac{1}{N_{t_i}(i)}\right) \|x_{T_k}\|_\infty + \frac{\gamma}{N_{t_i}(i)} \|x_{T_k}\|_\infty \\ &= \left[1 - \frac{1-\gamma}{N_{t_i}(i)}\right] \|x_{T_k}\|_\infty. \end{aligned} \quad (3)$$

Step 5: Replace $N_{t_i}(i)$ with the epoch-maximum N_k . The visit count at time t_i cannot exceed the maximum visits by the end of the epoch:

$$N_{t_i}(i) \leq N_{T_{k+1}}(i) \leq \max_j N_{T_{k+1}}(j) = N_k.$$

Since $1 - \gamma > 0$, the factor $\frac{1-\gamma}{N_{t_i}(i)}$ is at least $\frac{1-\gamma}{N_k}$, so

$$1 - \frac{1-\gamma}{N_{t_i}(i)} \leq 1 - \frac{1-\gamma}{N_k}.$$

Multiplying by the non-negative scalar $\|x_{T_k}\|_\infty$ preserves the inequality, thus from (3)

$$|x_{T_{k+1}}(i)| \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty. \quad (4)$$

Step 6: Take the maximum over all states. Inequality (4) holds for every state i . Therefore

$$\|x_{T_{k+1}}\|_\infty = \max_i |x_{T_{k+1}}(i)| \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty.$$

4.4 (8 Points) Polynomial-Decay Approximation

Assume the recursive bound from Section 4.2 holds exactly. Use the mathematical approximation for products,

$$\prod_{j=1}^k \left(1 - \frac{A}{j}\right) \approx k^{-A} \quad (\text{for small } 1/j),$$

to show that the error decays polynomially with respect to t . Conclude that:

$$\|x_t\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right),$$

where $R = p_{\min}/p_{\max}$.

Answer

Step 1: Epoch recursion and product form. From the epoch contraction bound we have, for each $k \geq 0$,

$$\|x_{T_{k+1}}\|_\infty \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty.$$

For the asymptotic analysis the inequality can be treated as an equality (the bound is tight in the worst case). Unrolling the recursion yields

$$\|x_{T_k}\|_\infty \leq \|x_0\|_\infty \prod_{j=0}^{k-1} \left(1 - \frac{1-\gamma}{N_j}\right). \quad (1)$$

Step 2: Estimate N_k via the law of large numbers. The states are sampled independently from a fixed distribution $p(i) > 0$. Define

$$p_{\min} = \min_i p(i), \quad p_{\max} = \max_i p(i), \quad R = \frac{p_{\min}}{p_{\max}} \in (0, 1].$$

By the law of large numbers, the time needed to complete k epochs grows linearly with k , i.e. $T_k \approx Ck$ for some constant $C > 0$. Moreover, the maximum number of visits to any state after k epochs is

$$N_k = \max_i N_{T_{k+1}}(i) \approx \frac{p_{\max}}{p_{\min}}(k+1) = \frac{k+1}{R}.$$

For sufficiently large k , $N_k \approx \frac{k}{R}$. Insert this into (1).

Step 3: Apply the product approximation. Retaining only the dominant terms, we have

$$1 - \frac{1-\gamma}{N_k} \approx 1 - \frac{R(1-\gamma)}{k} \quad (k \geq 1).$$

Thus for large k ,

$$\prod_{j=1}^{k-1} \left(1 - \frac{1-\gamma}{N_j}\right) \approx \prod_{j=1}^{k-1} \left(1 - \frac{R(1-\gamma)}{j}\right).$$

Using the given approximation $\prod_{j=1}^m \left(1 - \frac{A}{j}\right) \approx m^{-A}$ (valid for small $1/j$ and $A = R(1-\gamma)$ constant), we obtain

$$\prod_{j=1}^{k-1} \left(1 - \frac{R(1-\gamma)}{j}\right) \approx (k-1)^{-R(1-\gamma)} = \Theta\left(k^{-R(1-\gamma)}\right).$$

Therefore

$$\|x_{T_k}\|_{\infty} = O\left(k^{-R(1-\gamma)}\right). \quad (2)$$

Step 4: Convert epoch index k to real time t . Since T_k is proportional to k , we have $k \approx cT_k$ for some $c > 0$. Moreover, for any time t there exists k such that $T_k \leq t < T_{k+1}$, and because the sup-norm is non-increasing (see Step 2 of the epoch contraction proof),

$$\|x_t\|_{\infty} \leq \|x_{T_k}\|_{\infty}.$$

Combined with (2) and $k = \Theta(t)$, this gives

$$\|x_t\|_{\infty} = O\left(t^{-R(1-\gamma)}\right).$$

Hence the idealized process x_t decays polynomially with exponent $R(1-\gamma)$.

4.5 (3 Points) Noise-Dominance Condition

Suppose $z_t = O(t^{-(1-\gamma)})$ bounds the lower-bounding process, and the estimation noise satisfies $\epsilon_t = O(\sqrt{\log \log t/t})$. Assuming $\gamma > 1/2$, mathematically justify why ϵ_t decays strictly faster than z_t (and consequently faster than x_t).

Answer

Step 1: Asymptotic rates. We are given that the lower-bounding process z_t satisfies

$$z_t = O\left(\frac{1}{t^{1-\gamma}}\right).$$

The estimation noise $\epsilon_t = \|\hat{Q}_t - Q^*\|_\infty$ satisfies, by an extension of the Law of the Iterated Logarithm,

$$\epsilon_t = O\left(\sqrt{\frac{\log \log t}{t}}\right) = O\left(t^{-1/2} (\log \log t)^{1/2}\right).$$

Both bounds hold almost surely as $t \rightarrow \infty$.

Step 2: Compare the exponents. Because $\gamma > \frac{1}{2}$, we have

$$1 - \gamma < \frac{1}{2}.$$

Thus the polynomial exponent of z_t is $1 - \gamma$, which is strictly smaller than the exponent $\frac{1}{2}$ appearing in the noise bound.

Step 3: Subpolynomial growth of iterated logarithm. We need the standard fact that for any $\delta > 0$,

$$\lim_{t \rightarrow \infty} \frac{(\log \log t)^{1/2}}{t^\delta} = 0.$$

Proof sketch: Set $u = \log t$, then $\log \log t = \log u$. The ratio becomes $(\log u)^{1/2} / e^{\delta u}$. As $u \rightarrow \infty$, the exponential decays much faster than any power of a logarithm, so the limit is 0.

Hence the factor $(\log \log t)^{1/2}$ grows slower than t^δ for any $\delta > 0$. In particular it cannot compensate for a polynomial decay $t^{-(1/2-\gamma)}$ when $1/2 - \gamma > 0$.

Step 4: Show that ϵ_t decays strictly faster. Form the ratio

$$\frac{\epsilon_t}{z_t} = \frac{O(t^{-1/2}(\log \log t)^{1/2})}{O(t^{-(1-\gamma)})} = O(t^{1-\gamma-1/2} (\log \log t)^{1/2}).$$

Let $\delta = \frac{1}{2} - \gamma > 0$. Then

$$t^{1-\gamma-1/2} = t^{-\delta},$$

so the ratio is bounded by a constant times

$$\frac{(\log \log t)^{1/2}}{t^\delta} \rightarrow 0 \quad \text{as } t \rightarrow \infty,$$

by Step 3. Therefore

$$\lim_{t \rightarrow \infty} \frac{\epsilon_t}{z_t} = 0 \quad \text{almost surely.}$$

Thus $\epsilon_t = o(z_t)$: the noise decays strictly faster than z_t . Because x_t is upper-bounded by a process of the same order as z_t , the noise also decays strictly faster than x_t , and does not derail the convergence.

4.6 (1 Point) Noise-Term Meaning

In the perturbed epoch contraction

$$x_{T_{k+1}+1}(i) \leq \left(1 - \frac{1-\gamma}{S_k}\right) \|x_{T_k+1}\| + \frac{\gamma}{s_k} \epsilon_{T_k},$$

what is the meaning of the term $\frac{\gamma}{s_k} \epsilon_{T_k}$? Why does it get divided by s_k ?

Answer

The term $\frac{\gamma}{s_k} \epsilon_{T_k}$ quantifies the **residual error** that the auxiliary process introduces into the main process after a full epoch, despite the deterministic contraction. In each individual update, the error ϵ_{T_k} enters multiplied by the step-size factor $\gamma/N_t(i)$. Over the epoch, a state i is updated at least s_k times. Summing the noise contributions over these updates and applying the contraction, the worst-case cumulative effect is bounded by something proportional to $\frac{\gamma}{s_k} \epsilon_{T_k}$.

Why divide by s_k ? The division by s_k reflects **averaging** of the noise over multiple visits. Even if the instantaneous noise ϵ_{T_k} stays constant throughout the epoch, repeated updates dilute its impact. Because the learning rate shrinks with the visit count, each noise injection is weighted by approximately $1/(\text{visit number})$, and the total effect across s_k updates behaves like a harmonic sum, which is on the order of ϵ_{T_k}/s_k . Thus, the more frequently every state is visited (larger s_k), the more the noise is averaged out, and the smaller its contribution to the post-epoch error. This is crucial for stability: it guarantees that the stochastic noise does not overwhelm the deterministic contraction as the algorithm runs longer and the number of visits grows.

4.7 (6 Points) Main Theorem Proof

Prove that the Q -learning error satisfies both bounds:

$$\|Q_t - Q^*\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right), \quad \|Q_t - Q^*\|_\infty = O\left(\sqrt{\frac{\log \log t}{t}}\right).$$

Answer

The total error of Q_t relative to Q^* is split using the triangle inequality:

$$|Q_t(x, a) - Q^*(x, a)| \leq \underbrace{|Q_t(x, a) - \hat{Q}_t(x, a)|}_{\Delta_t(x, a)} + \underbrace{|\hat{Q}_t(x, a) - Q^*(x, a)|}_{\leq \epsilon_t},$$

where $\epsilon_t = \|\hat{Q}_t - Q^*\|_\infty$. Hence

$$\|Q_t - Q^*\|_\infty \leq \|\Delta_t\|_\infty + \epsilon_t. \quad (1)$$

We now bound the two terms on the right.

Bound on $\|\Delta_t\|_\infty$. The error process Δ_t follows the recursion studied in Section 4.1. After simplifying by ignoring the noise term and analysing epochs, we obtained the abstract process x_t whose sup-norm contracts every epoch (Section 4.2) and decays as

$$\|x_t\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right) \quad (\text{Section 4.3}).$$

Because Δ_t is upper-bounded by x_t (up to a constant factor once the noise is absorbed), the same polynomial bound holds:

$$\|\Delta_t\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right). \quad (2)$$

Bound on ϵ_t . The auxiliary process $\hat{Q}_t$ estimates the stationary target Q^* using sample averages. By an extension of the Law of the Iterated Logarithm (LIL),

$$\epsilon_t = O\left(\sqrt{\frac{\log \log t}{t}}\right). \quad (3)$$

Combining the bounds. Insert (2) and (3) into (1):

$$\|Q_t - Q^*\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right) + O\left(\sqrt{\frac{\log \log t}{t}}\right).$$

Both terms are non-negative, so there exists a constant $B > 0$ such that each of them bounds the total error individually (asymptotically and almost surely). More precisely, for all sufficiently large t ,

$$\|Q_t - Q^*\|_\infty \leq B \cdot \frac{1}{t^{R(1-\gamma)}}, \quad \|Q_t - Q^*\|_\infty \leq B \cdot \sqrt{\frac{\log \log t}{t}}.$$

Thus the two bounds hold. $\square$

4.8 (1 Point) Threshold for Rates

Under what condition on $p_{\min}$, $p_{\max}$, and γ does the first bound dominate?

Answer

Which rate dominates? The first bound is a pure polynomial $t^{-R(1-\gamma)}$, while the second is of order $t^{-1/2}(\log \log t)^{1/2}$. For large t , the one with the smaller exponent of t dominates (decays more slowly). The exponent of the first bound is $R(1-\gamma)$, and the exponent of the second is $\frac{1}{2}$.

Thus the first bound is the slower (dominant) one precisely when

$$R(1-\gamma) \leq \frac{1}{2}.$$

Express the condition in terms of γ and $p_{\min}, p_{\max}$. Recall $R = p_{\min}/p_{\max}$. Substituting:

$$\frac{p_{\min}}{p_{\max}}(1-\gamma) \leq \frac{1}{2} \iff 1-\gamma \leq \frac{p_{\max}}{2p_{\min}} \iff \gamma \geq 1 - \frac{p_{\max}}{2p_{\min}}.$$

When this inequality holds, the polynomial bound $t^{-R(1-\gamma)}$ decays more slowly than the LIL bound. Consequently, the overall convergence rate is limited by the polynomial term, and the first bound is the dominant one. $\square$

4.9 (1 Point) Convergence Speed as $\gamma \rightarrow 1$

In reinforcement learning, a discount factor γ close to 1 is typically used for tasks with long horizons. Based on this theorem, what happens to the theoretical convergence speed of Q -learning as the task horizon increases ($\gamma \rightarrow 1$)?

Answer

From the theorem, the error decays at the slower of the two rates

$$\frac{1}{t^{R(1-\gamma)}} \quad \text{and} \quad \sqrt{\frac{\log \log t}{t}}.$$

As $\gamma \rightarrow 1$, the exponent $R(1 - \gamma) \rightarrow 0$. Hence the first bound behaves roughly like $t^{-R(1-\gamma)} \approx t^0 = 1$ for large t , meaning it decays extremely slowly. The second bound is independent of γ (provided $\gamma > \frac{1}{2}$) and decays like $t^{-1/2}$ up to a logarithmic factor, which is far faster than a near-constant.

Thus, for γ very close to 1, the polynomial rate dominates and becomes arbitrarily slow: the theoretical convergence speed of Q -learning degrades dramatically as the effective planning horizon lengthens. In the limit $\gamma \rightarrow 1$, the error is essentially bounded by a constant multiple of $1/t^{R(1-\gamma)}$, which can be orders of magnitude slower than the $1/\sqrt{t}$ rate that would hold for smaller γ . This illustrates a fundamental tension between long-horizon credit assignment (requiring $\gamma \approx 1$) and sample efficiency.

4.10 (1 Point) Uniform-Sampling

Suppose a practitioner ensures a perfectly uniform exploration strategy such that every state-action pair is sampled with strictly equal probability ($p_{\min} = p_{\max}$). If they set $\gamma = 0.9$, which rate dictates the convergence speed, and what is the exact polynomial exponent of the decay?

Answer

With a perfectly uniform exploration strategy, every state-action pair is sampled with equal probability, so

$$p_{\min} = p_{\max}, \quad R = \frac{p_{\min}}{p_{\max}} = 1.$$

Given $\gamma = 0.9$, we compute the exponent of the polynomial bound:

$$R(1 - \gamma) = 1 \times 0.1 = 0.1.$$

The two possible rates are

- polynomial: $t^{-0.1}$
- LIL: $t^{-0.5} \sqrt{\log \log t}$.

Because $0.1 < 0.5$, the polynomial rate decays far more slowly and therefore dictates the overall convergence speed. The exact polynomial exponent of the dominant decay is **0.1**, so the error satisfies $\|Q_t - Q^*\|_{\infty} = O(t^{-0.1})$ almost surely.